Lokasamyoga - Connecting Hearts, Changing Lives

Final Evaluation Project Report

1. Project Objective

Lokasamyoga is a centralized digital platform designed to bridge the gap between NGOs, Volunteers, and Donors. It simplifies social campaign management, streamlines volunteer participation, and facilitates transparent donation communication to amplify real-world social impact.

2. Technology Stack & Architecture

- **Frontend Engine**: Built natively using **React.js (Vite)** bundled tightly with **TypeScript** to achieve a strict, high-performance web architecture.
- **Styling Architecture**: Custom CSS featuring advanced UI/UX principles like **Glassmorphism**, floating layout widgets, deep CSS Variables, and animated micro-interactions without relying heavily on unoptimized external component libraries.
- **Iconography**: Clean vector visuals supplied directly via `lucide-react`.
- **Database (Evaluation Prototype)**: Instead of mocking a dummy backend, the application utilizes the browser's native **LocalStorage API** to act identically to a NoSQL JSON database (like MongoDB). This advanced mechanism allows the app to genuinely "save" state. E.g., a volunteer creates an application -> it saves to the Database -> an NGO logs in moments later -> reads the Database securely -> sees the new application!
- **Routing**: Client-side logic handled flawlessly by `react-router-dom`.

3. Core System Flow & User Roles

🏢 NGO (Non-Governmental Organization)

Purpose: Manage active campaigns and organize mass recruitments.

- **Analytics Dashboard**: Tracks key metrics natively—Total Campaigns, Approved Volunteers, and Total Funds Received (₹).
- **Campaign Engine**: The NGO can actively create detailed campaigns by submitting custom Titles, Dates, Geographical Locations, Required Volunteer Caps, extensive Paragraph Descriptions, and striking Cover Photo URLs.
- **Application Management**: Clicking "Manage" beside an active campaign securely loads a live table of pending Volunteer Applications. NGOs can exclusively **Accept** or **Reject** these applications.

🙋‍♂️ Volunteer

Purpose: Readily discover and physically participate in social drives.

- **Analytics Dashboard**: Personal metrics tracking total Hours Volunteered and specific Campaigns Attended.
- **Visual Discovery Portal**: Volunteers dynamically browse "Available Campaigns" using an image-rich Grid matrix.
- **Intelligent Detail Views**: Clicking "View Details" actively queries the exact Campaign structure injected by the NGO, showing the volunteer the custom Cover Photo, full description, and location constraints before executing a secure "Apply Now" action.
- **Status Tracking**: The data table actively listens to the local database to update their application states automatically from *Pending* to *Approved* or *Rejected*.

💰 Donor

Purpose: Securely funnel financial resources to valid initiatives.

- **Analytics Dashboard**: Logs Total Donated Globally (₹) tracking all financial movements exclusively tied to that active session.
- **Direct Interaction**: Donors can actively identify Verified NGOs on the platform and trigger a Donation Modal.
- **Transaction Logging**: Initiating a transaction requires specifying actual Amounts and secure Gateways (UPI/Cards). Upon successful transaction, the value immediately maps to their secure permanent "Donation History Log".

🛡️ System Admin

Purpose: Top-down control over platform security and validation.

- **User Oversight**: Admins review a database list mapping out all registered system users (NGOs & Volunteers). They hold executive power to securely **Ban Accounts**, actively neutralizing fake or spam networks.
- **Registration Verification**: Admins control the flow of pending NGO registrations (Environment, Education, etc.), having explicit permission to Approve or Reject initial onboarding requests.

4. Viva Tips: How to Explain This Project

If your evaluator asks you questions, here are the best ways to frame your answers:

1. **"How does the Data persist without a live server?"** *Answer*: "For this evaluation prototype, I engineered a custom LocalStorage JSON abstraction layer. Every time a user interacts (Creates a Campaign, Approves a Volunteer), the system intercepts the component state, stringifies it, and forcefully writes it to the browser's secured `localStorage`. When the app routes to a new dashboard, it reads exclusively from this unified local database string, creating a perfect simulation of a live asynchronous NoSQL fetch operation."
2. **"What differentiates this from standard React projects?"** *Answer*: "The highly restrictive, dynamic Role-Based UI handling. We isolated four drastically different user experiences (NGO vs Volunteer vs Donor vs Admin) operating over one singular frontend component tree. The UI intelligently strips or reveals components depending entirely on the authorization role context securely passed during the Login phase."
3. **"What makes the UI so unique?"** *Answer*: "The styling is built utilizing 'Glassmorphism' against Deep Dark-Mode palettes heavily dependent on transparent RGBA values, CSS background blurring, and carefully designed Box-Shadow gradients. It looks like a premium web3 desktop application rather than a basic web form."